The Geometry of Natality: Creation-Enabled Reachability and Cost-Optimal Repair in Dependency Graphs

Mohammad Amir Khusru Akhtar
Usha Martin University, Ranchi, Jharkhand, India

Abstract

Repair in a dependency network may require more than restoring an existing vertex: a replacement must be introduced, made operational, and connected to the services that depend on it. We study this distinction through a creation-enabled, costed transition system for evolving dependency graphs. Its states retain component identity, operational status, dependencies, and action history; admissible transitions include restoration, creation, and rewiring. A directed extended path-cost function makes the reachability and optimization questions precise without presuming metric symmetry. We establish elementary but informative separation and cost-comparison statements for creation-enabled versus restoration-only planning, and state the assumptions under which independent repair subproblems decompose exactly. A three-service instance admits two minimum-cost strategies of cost three. Across 12 synthetic search instances, exhaustive enumeration and uniform-cost search agree on the optimum; for four independent groups, enumeration visits 314,217 search nodes and uniform-cost search expands 399 states. A further 24 synthetic cases exercise guarded decomposition, including a shared-capacity counterexample to unqualified plan composition. We also formalize fresh-name equivalence and prove cost preservation for equivariant identity quotients, while separating a small executable symmetry check from unperformed comparative experiments. These findings provide an explicit formal vocabulary and reproducible correctness evidence for creation-aware recovery; they do not establish a new graph-transformation calculus or superiority over established planning and verification tools.

Keywords: dependency graph; graph transformation; cost-optimal repair; service recovery; component creation; state-space search; decomposition

1 Introduction

Repairing a failed dependency is not always a matter of bringing the failed component back. A replacement may be created, yet the system remains unavailable until its dependents are redirected. The cost of recovery therefore belongs not to an isolated repair operation but to a sequence of changes in a graph. This distinction motivates the *geometry of natality*: a view of repair in which the creation of components is represented explicitly, alongside the relationships that make the new components operational.

Graph transformation provides established machinery for changing and analyzing structured states [1, 2]; GROOVE makes this machinery executable for state-space exploration [3, 4]. Abstraction, state-space reduction, and graph-based verification have long histories [5, 6, 7, 8]. Consequently, creation-aware graph planning cannot be treated as novel merely because its states contain newly created vertices. The question addressed here is more specific: what can be established, by exact search and independent checks, about cost-optimal repair when a failed component can be restored or replaced?

We make three contributions. First, we define a finite repair-planning model that keeps graph structure, action history, and recovery conditions explicit. Second, we construct a small auditable recovery instance and a parameterized family in which two independent exact procedures agree

on minimum cost. Third, we identify a precise decomposition boundary: independent local repairs compose optimally, whereas shared restrictions can invalidate locally selected plans. The analysis separates what follows from the availability of creation actions from what follows solely from standard shortest-path and decomposition principles.

2 Background and relation to established methods

Dependency graphs are a natural representation of systems in which the availability of one component depends on another; graph structure and network function need not coincide [9]. Graph-rewriting rules can represent changes to both nodes and edges [1, 2, 10]. The distinguishing modelling decision in this work is to expose replacement creation and subsequent rewiring as separate, costed actions rather than collapse them into a single abstract recovery event.

The choice has consequences for search. Two configurations with the same number of live components may differ in which dependency is satisfied. Furthermore, two states with the same graph may admit different future actions when rules have history-dependent prerequisites. For exact exploration, the state key must therefore retain all information that affects future applicability and the recovery predicate. The comparison in Table 1 situates the present work without asserting superiority over existing tools.

Table 1: Scope of the present formulation relative to established approaches.

Approach	Established capability	Relationship to this study
Graph transformation [1, 2]	Rule-based changes to graph structure	Provides the underlying conceptual vocabulary
GROOVE [4]	Executable exploration of graph-transformation systems	Relevant future matched implementation; no matched results claimed
State-space reduction [6, 8]	Reduction and abstraction of verification spaces	Prevents attributing generic reduction ideas to natality
Present exact search	Explicit costs, action prerequisites, replacement creation, and recovery goals	Verified on synthetic instances; no new search complexity guarantee

2.1 Related structural and optimization perspectives

Classical depth-first graph algorithms clarify the role of connectivity and structural decomposition in dependency analysis [11]; network topology alone does not specify recovery feasibility [9]. The distinction between graph structure and an operational objective also appears in influence maximization [12] and submodular optimization [13], although neither directly solves the action-sequencing problem considered here. Graph augmentation [14] changes connectivity, whereas precedence-constrained optimization formalizes choices subject to dependency restrictions [15, 16, 17]. These are adjacent problems, not substitutes for a stateful repair planner.

Name generation in nominal rewriting [18] and graph passing [19] provide formal context for newly introduced graph objects. Confluence analysis [20] concerns the relationships among rewriting paths; model-transformation validation [21] and learning-based graph-transformation testing [22] investigate different verification strategies. GROOVE has also supported reaction-system experiments [23]. These studies motivate separating representational adequacy, optimality of the search procedure, and empirical comparison of tools. They do not establish a performance advantage for the present implementation.

3 Creation-enabled state space and operational geometry

A repair system is described by a tuple

$$\mathcal{N} = (\mathcal{S}, \mathcal{A}, T, c, \Gamma, \Phi), \tag{1}$$

where $\mathcal{S}$ is a set of identity-bearing attributed dependency graphs with action histories, $\mathcal{A}$ is a set of typed actions, T is a partial transition map, $c : \mathcal{A} \to \mathbb{R}_{\geq 0}$ assigns costs, $\Gamma \subseteq \mathcal{S}$ identifies admissible states, and $\Phi \subseteq \Gamma$ is the recovery goal. The general model may have infinitely many states when creation draws fresh identities from an unbounded supply. The exact-search claims below apply only to finite instances with a fixed finite action alphabet, bounded fresh identities, and single-use actions; the experiments satisfy these restrictions. The identity of a newly created component is distinct from the identity of the failed component even when both implement the same service role. This matters because dependencies refer to particular vertices, not merely to service labels.

We distinguish three kinds of transition. A *restoration* changes the operational attributes of an existing vertex without introducing a new identity. A *creation* introduces a fresh vertex identity into the graph; the newly created component need not yet serve any dependent. A *rewiring* changes dependency edges without introducing a new vertex. These types are modelling distinctions, not claims that existing graph-rewriting formalisms lack the ability to represent them [1, 2, 18].

Let $\mathcal{A}_0$ contain all admissible creation-free actions, including restoration and rewiring, and let $\mathcal{A}_+$ additionally permit creation. Thus the restricted model is creation-free, not necessarily restoration-only. For a fixed initial state s, write $\mathcal{R}_0(s)$ and $\mathcal{R}_+(s)$ for the states reachable by valid finite paths under the respective action sets. A directed extended operational distance is

$$d_+(s,t) = \inf_{\substack{\pi : s \rightsquigarrow t \\ \pi \text{ admissible}}} \sum_{a \in \pi} c(a), \qquad \inf \varnothing = +\infty. \tag{2}$$

The same definition with paths restricted to $\mathcal{A}_0$ gives d_0. Concatenation of admissible paths yields the directed triangle inequality whenever the relevant paths exist. In general $d_+(s,t) \neq d_+(t,s)$, and distinct states may be connected at zero cost; this construction is therefore not asserted to be a metric. Its value lies in making operational reachability and cost explicit rather than assigning an unsupported geometric interpretation to graph drawings.

Proposition 1 (Reachability and cost monotonicity under added actions)**.** *Assume the two models have identical initial states, common-action transitions, costs, admissibility constraints, and goals, and differ only because $\mathcal{A}_0 \subseteq \mathcal{A}_+$. Then $\mathcal{R}_0(s) \subseteq \mathcal{R}_+(s)$, and the optimal goal-reaching costs satisfy $\mathrm{Opt}_+(s) \leq \mathrm{Opt}_0(s)$, with $+\infty$ allowed.*

Proof. Every path using only $\mathcal{A}_0$ is also an admissible path in the expanded system, by the common-transition and common-constraint assumptions. Reachable-state inclusion follows. Taking the infimum over a superset of feasible goal-reaching paths cannot increase its value. □

The inequality can be strict, but need not be. For example, if restoration costs five units and creation followed by rewiring costs three, with both routes reaching the same recovery goal, then $\mathrm{Opt}_+ = 3 < 5 = \mathrm{Opt}_0$. This is a constructed example illustrating the proposition, not a measured result. In the evaluated fixture, both routes cost three, so creation does not strictly improve the optimum.

Proposition 2 (Creation-essential recovery)**.** *Under the assumptions of Proposition 1, creation is necessary for reaching a goal from s exactly when $\mathcal{R}_0(s) \cap \Phi = \varnothing$ and $\mathcal{R}_+(s) \cap \Phi \neq \varnothing$.*

Proof. The first condition states that no creation-free admissible path reaches a goal; the second exhibits a goal-reaching path in the expanded system. Conversely, if a goal is reachable only through a path that uses creation, it cannot belong to $\mathcal{R}_0(s)$ but belongs to $\mathcal{R}_+(s)$. □

For a concrete separation, consider a failed service whose restoration action is forbidden by policy, while creation of a replacement and redirection of its dependent are permitted. The creation-free goal set is unreachable; the creation-enabled system reaches the goal after the two permitted operations. This small construction isolates a difference in *feasible action sets*, not a representational impossibility for established graph-transformation systems. The propositions are direct consequences of transition-system inclusion, and no novelty is claimed for the underlying mathematical principle.

Table 2: Operational distinctions in the creation-enabled model.

Operation	Identity and structure	Recovery implication
Restoration	Preserves vertex identity; changes attributes	Can restore existing dependencies
Creation	Adds a fresh vertex identity	Does not itself redirect dependents
Rewiring	Preserves vertex set; changes edges	Can connect dependents to a replacement
Combined plan	May change identities, attributes, and edges	Must satisfy the same goal and constraints

The distinction between these operations is summarized in Table 2. It permits explicit tests of whether the ability to create components changes feasible recovery paths or their minimum cost.

4 Finite repair states, actions, and objectives

Let a dependency graph be $G = (V, E, \ell)$, where $(u, v) \in E$ means that component u depends on component v, and $\ell(v)$ records whether v is operational. A repair state is $s = (G, H)$, where $H \subseteq A$ records actions already performed. In the evaluated single-use rules, the set H is sufficient: prerequisites test whether a named action has occurred, not the order of multiple prior actions. For genuinely order-sensitive policies, the state must instead retain an appropriate ordered history or sufficient automaton state. A finite action $a \in A$ has a partial transition T_a, an applicability predicate P_a, and nonnegative cost $c(a)$. Each action is permitted at most once in the experimental model.

A repair plan $\pi = (a_1, \ldots, a_k)$ is valid when each action is applicable in sequence, no forbidden configuration is encountered, and the total cost does not exceed budget B. The objective is

$$\mathrm{Opt}(s_0) = \min_{\pi:\, \mathrm{Valid}(\pi, s_0),\, \mathrm{Goal}(T_\pi(s_0))} \mathrm{Cost}(\pi), \qquad \mathrm{Cost}(\pi) = \sum_{j=1}^{k} c(a_j). \tag{3}$$

We use $\mathrm{Opt}(s_0) = +\infty$ if no feasible goal-reaching plan exists. In the simple fixture, a component is usable if it is operational and every component on its recursively traversed dependency chain is usable; cyclic chains are treated as unusable by the fixture oracle. This convention is specific to the experimental recovery predicate, not a universal semantics for dependency cycles.

The finite implementation instantiates the general state-space definition in Equation (1). The state transition structure is summarized in Figure 1; the concrete action semantics appear in Table 3.

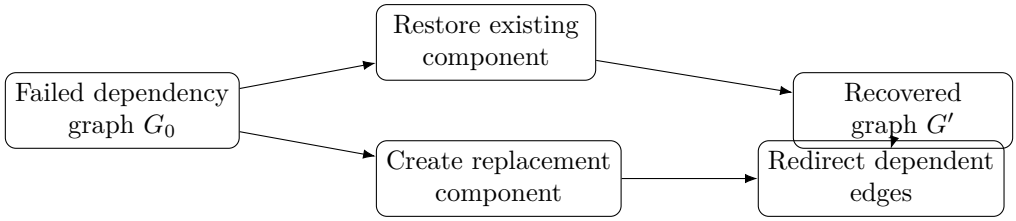


Figure 1: Two recovery routes: restoration of an existing component, or creation followed by rewiring. Each arrow is a costed action; the lower route requires both operations.

Table 3: Actions in the three-service synthetic recovery instance.

Action	Cost	Prerequisite	State change
Restart authentication	3	None	Marks the original authentication service operational
Create replacement	2	None	Adds an operational replacement authentication service
Rewire reporting	1	Replacement created	Replaces the reporting-to-original edge with a reporting-to-replacement edge

5 Exact search and verification

We use two complementary exact procedures. Exhaustive enumeration supplies a finite audit trail for a small action set, while uniform-cost search provides an exact shortest-path baseline over the reachable state graph. Their agreement checks the model and implementation without implying that either algorithm is new.

Algorithm 1 enumerates ordered subsets of distinct actions. Because every action occurs at most once, the number of candidate sequences before feasibility pruning is $\sum_{j=0}^{|A|} |A|!/(|A|-j)!$.

Algorithm 1 Exhaustive finite repair oracle

Require: Initial state s_0, actions A, budget B, goal predicate
$W \leftarrow \varnothing$
for each ordered subset π of distinct actions from A **do**
  Simulate π from s_0 in order
  if all actions are applicable, constraints hold, and $\text{Cost}(\pi) \leq B$ **then**
    if $\text{Goal}(T_\pi(s_0))$ **then**
      $W \leftarrow W \cup \{(\text{Cost}(\pi), \pi)\}$
    end if
  end if
end for
return the minimum-cost entries in W, or infeasible if $W = \varnothing$

Algorithm 2 uses a priority queue and a key containing both graph state and used-action history. Omitting history would be unsound for the prerequisite rules used here.

Algorithm 2 Uniform-cost repair search

Require: Initial state s_0, finite actions A, nonnegative costs
$d[s_0] \leftarrow 0$; initialize priority queue $Q \leftarrow \{(0, s_0)\}$
while $Q \neq \varnothing$ **do**
    Remove minimum-cost pair (g, s) from Q
    if $g \neq d[s]$ **then**
        continue
    end if
    if Goal(s) **then**
        return g
    end if
    for each applicable unused action a **do**
        $s' \leftarrow T_a(s)$; $g' \leftarrow g + c(a)$
        if s' is permitted and $g' < d.get(s', +\infty)$ **then**
            $d[s'] \leftarrow g'$; insert (g', s') into Q
        end if
    end for
end while
return infeasible

Proposition 3. *For a finite transition system with nonnegative action costs and a state key that preserves all future applicability, transition costs, and goal outcomes, Algorithm 2 returns the minimum goal-reaching cost whenever one exists.*

Proof. The transition system is a finite directed graph with nonnegative edge weights. Uniform-cost search is Dijkstra's shortest-path algorithm on that graph. Its relaxation invariant ensures that a goal state removed from the queue at its current best distance cannot be improved by an undiscovered path of smaller cost. Recording history in the key prevents the identification of states with different future action sets. □

6 The recovery fixture

The initial graph contains an operational ingress service, a failed authentication service, and an operational reporting service. Reporting depends on authentication and is therefore unusable until that dependency is repaired. The two successful minimum-cost paths are shown in Figure 2 and listed in Table 4.

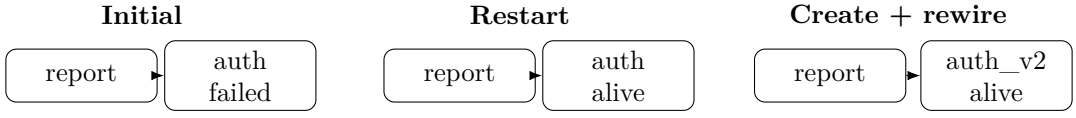


Figure 2: Dependency edge before recovery and after the two alternative optimal repairs. The ingress component, unchanged in all three configurations, is omitted for clarity.

Table 4: Exact recovery results for the three-service fixture.

Action sequence	Total cost	Goal reached?	Optimal?
Restart authentication	3	Yes	Yes
Create replacement; rewire reporting	3	Yes	Yes
Create replacement only	2	No	No
No action	0	No	No

An exhaustive certificate enumerates all $1 + 3 + 6 + 6 = 16$ ordered subsets of the three actions. Exactly four sequences pass the fixture's applicability and budget checks; two reach the goal, both at cost three. The independent exact procedures agree on this optimum. This is a correctness test for the encoded rules, not evidence that creation always reduces repair cost.

7 Synthetic scaling study

To test search behavior beyond one example, we construct a family of n independent failed-service groups. Each group offers a restart of cost three, or creation of cost two followed by rewiring of cost one. The global goal requires recovery of every group. Three deterministic action-order seeds are used for each $n \in \{1, 2, 3, 4\}$, giving 12 executions over four underlying graph sizes. These seeds change action ordering, not graph topology or repair costs. Both exact methods must return $3n$. The experimental design is summarized in Table 5.

Table 5: Experimental design and checks for the synthetic search comparison.

Property	Setting
Repair groups	$n = 1, 2, 3, 4$
Actions per group	Three: restart, create, rewire
Action-order seeds	$0, 1, 2$ for each n
Total instances	12
Methods	Exhaustive enumeration and uniform-cost search
Correctness check	Exact agreement on minimum cost $3n$
Reported search metric	Nodes visited by exhaustive enumeration; states expanded by uniform-cost search
Scope	Synthetic independent groups; no matched GROOVE experiment

The results in Table 6 show that uniform-cost search explores substantially fewer nodes than enumeration in the reported cases. The methods do not measure precisely identical quantities: enumeration counts visits in its search tree, including repeated configurations, while uniform-cost search counts settled state expansions. The difference is meaningful for the implemented algorithms but must not be interpreted as an advantage over GROOVE or as a novel reduction result.

Table 6: Selected deterministic search counts from the successful automated experiment. Each listed count is reproduced across the three action-order seeds.

Groups n	Actions	Optimal cost	Enumeration visits	UCS expansions
3	9	9	3,796	78
4	12	12	314,217	399

Figure 3 plots these counts on a logarithmic axis to retain both series on a readable scale. No inference about asymptotic complexity is drawn from two displayed sizes.

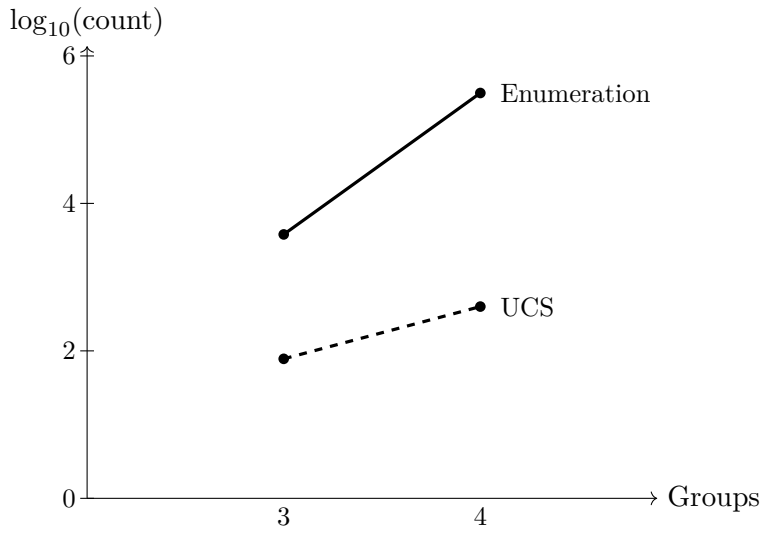


Figure 3: Search effort for three and four independent repair groups on a logarithmic plotting scale. Points represent counts from Table 6, not runtime or memory.

8 Exact decomposition and its boundary

The synthetic family admits an additional analysis. If local actions cannot influence other groups, the global optimization problem separates. The decomposition is mathematically elementary, but its conditions provide a useful check against unjustified reduction.

Proposition 4. *Suppose the global state is the product of n local states; each action changes exactly one local state, with applicability and cost depending only on that local state; the goal is the conjunction of local goals; and no cross-group safety or resource restriction applies. Then*

$$\mathrm{Opt}(s_0) = \sum_{i=1}^{n} \mathrm{Opt}_i(s_{0,i}). \tag{4}$$

Moreover, concatenating one optimal plan for each local problem produces a globally optimal plan. The evaluated implementation reports only the optimum cost, not the concatenated plan.

Proof. Project any feasible global plan onto the actions of group i. Locality ensures that each projection is a feasible plan for that group's goal. By additivity, the global plan costs at least the sum of local optima. Conversely, concatenate optimal local plans. Their actions commute with those of other groups in the sense of preserving applicability and outcomes, so the concatenation reaches every local goal and attains the lower bound. □

Algorithm 3 implements the guarded decomposition. The guard is part of the method: a cost identity derived under independence cannot be used without checking independence. In the evaluated code, declared coupling edges or a global resource limit trigger rejection; this is not a general automated proof of semantic independence. The sufficient syntactic certificate developed below explains what a broader implementation would have to check.

Algorithm 3 Guarded component-wise exact repair

Require: Repair instance I, proposed partition $\{I_1, \ldots, I_n\}$
1: **if** an action, goal, constraint, or resource couples different groups **then**
2: **return** "decomposition not certified"
3: **end if**
4: **for** $i \leftarrow 1$ to n **do**
5: $c_i \leftarrow \textsc{UniformCostRepairCost}(I_i)$
6: **end for**
7: **return** $\sum_i c_i$

8.1 A sufficient syntactic independence certificate

For a finite, explicitly factored state description, let $X = \dot{\bigcup}_{i=1}^{n} X_i$ be disjoint sets of state variables. For every action a, let $R(a)$ and $W(a)$ denote conservative read and write footprints, including every variable used by its precondition, effect, or action-dependent cost. A partition is *syntactically certified* if each action belongs to exactly one group i with $R(a) \cup W(a) \subseteq X_i$, every goal predicate factors as $\bigwedge_i \Phi_i(X_i)$, and every admissibility or resource predicate factors into local predicates. Any unclassified global resource, cross-component edge, or uncertain footprint causes rejection.

Proposition 5 (Soundness of a local-footprint certificate)**.** *For a finite transition system with complete conservative action footprints, local costs, a product initial state, and factorized goals and admissibility, acceptance of the syntactic certificate implies the hypotheses of Proposition 4. Consequently the sum of local optimum costs is the global optimum whenever each local problem is feasible.*

Proof. Because the read and write footprints of every action are contained in one variable block, executing it cannot change another block or depend on another block's value. Complete footprints include all cost and applicability dependencies. Factorized admissibility and goals prohibit hidden cross-block restrictions. Thus every feasible global action sequence projects to feasible local sequences, and any concatenation of feasible local sequences remains admissible. The two inequalities in the proof of Proposition 4 therefore apply. □

The certificate is conservative: rejection does not imply that decomposition is impossible. The certificate and Proposition 5 are an analytical sufficient condition, *not* a claim that the published experimental code extracts read/write sets or implements a general-purpose static analyzer. Shared restart capacity violates factorization and is correctly outside the certificate's scope.

Twenty-four synthetic decomposition executions were completed, using $n = 1, \ldots, 8$ and three action-order seeds; the seeds do not generate independent dependency topologies. Every case returned the expected total cost $3n$; for $n \leq 4$, full-state uniform-cost search independently agreed. Table 7 distinguishes these checks from the unperformed large-instance full-state comparisons.

Table 7: Scope of the decomposition validation.

Groups	Decomposition check	Independent full-state check
1–4 (12 cases)	Cost $3n$ in all cases	UCS agreement in all cases
5–8 (12 cases)	Cost $3n$ in all cases	Not performed
Two coupled groups	Coupling guard rejects partition	Independent restart–restart plan violates shared capacity

The coupling counterexample is concrete. Consider two groups with a shared capacity permitting at most one restart. Independent local optimization can choose a restart for each group, producing a cost-six plan that is infeasible under the shared constraint. Alternative creation-and-rewiring choices can retain the same numerical optimum in this example; equality of objective values does not certify the validity of the returned plan. The distinction between preserving cost and preserving solutions is illustrated in Figure 4.

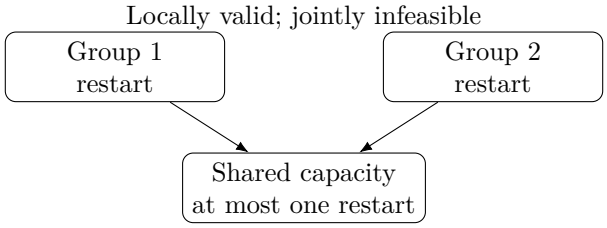


Figure 4: A shared restart limit breaks the noninterference assumption. A partition guard must reject the naive concatenation of locally chosen repair plans.

9 Fresh identities, equivariance, and exact quotient search

A replacement identity is operationally relevant when a dependency points to it, but its arbitrary spelling need not be relevant. Let I_0 be the finite set of identities present initially, and let F be a bounded set of fresh identities. A permutation σ of F fixes I_0 and acts simultaneously on vertex names, edge endpoints, attributes containing identities, and history-relevant facts. Write σs for the resulting state. Two states are *fresh-name equivalent*, $s \sim_N t$, if $t = \sigma s$ for such a permutation. Only well-typed permutations preserving service roles are permitted.

A transition system is *equivariant* if each admissible transition $s \xrightarrow{a,w} t$ has a corresponding transition $\sigma s \xrightarrow{\sigma a,w} \sigma t$ and conversely, where w is the action cost. Its goal predicate and safety constraints must be invariant under these permutations. In particular, negative application conditions, capacity reservations, and allocation policies must be preserved. Name-dependent prices, hard-coded identity tests, or global fresh-name counters can invalidate the hypothesis. History is renamed along with the graph, rather than discarded.

Proposition 6 (Fresh-name quotient preserves optimal recovery)**.** *For a finite, nonnegative-cost, equivariant repair transition system with permutation-invariant goals and admissibility, the orbit quotient $\mathcal{S}/\sim_N$ preserves goal reachability and the minimum goal-reaching cost from every initial state. A quotient path can be lifted to a concrete path with the same cost.*

Proof. Each transition $s \xrightarrow{a,w} t$ induces an orbit transition $[s] \xrightarrow{w} [t]$. This is well defined: for any other representative $s' = \sigma s$, equivariance supplies $s' \xrightarrow{\sigma a,w} \sigma t$, whose target is in $[t]$. Conversely, an orbit transition from $[s]$ may be represented by a transition starting at an arbitrary chosen representative of $[s]$ by applying the inverse renaming. Repeating this lifting step constructs a concrete path for every quotient path, with the same sequence of weights. Projection of concrete paths gives quotient paths with unchanged weights. Goal and admissibility invariance ensure that projection and lifting preserve feasibility and success. The sets of attainable goal-reaching path costs therefore coincide, so their minima coincide. □

The proposition is an application of established symmetry and nominal reasoning, not a claim that quotient-state optimality was first discovered here [18, 5, 6, 8]. Its value for the repair model is to specify exactly which identity-bearing facts must survive renaming. A coarse count of live vertices, edges, and creations is not an adequate substitute for an orbit key: counts can agree even when the dependency structure and future reachability differ.

Algorithm 4 Exact search over certified fresh-name orbits

Require: Finite equivariant repair system, initial state s_0, canonical orbit key K

$d[K(s_0)] \leftarrow 0$; $Q \leftarrow \{(0, s_0)\}$
while $Q \neq \varnothing$ **do**
    Remove minimum-cost pair (g, s) from Q
    if $g \neq d[K(s)]$ **then**
        continue
    end if
    if Goal(s) **then**
        return g
    end if
    for each admissible successor s' of s with cost w **do**
        if $g + w < d.get(K(s'), +\infty)$ **then**
            $d[K(s')] \leftarrow g + w$; insert $(g + w, s')$ into Q
        end if
    end for
end while
return infeasible

Algorithm 4 is correct under the hypotheses of Proposition 6, provided $K(s) = K(t)$ exactly for states in the same permitted orbit. Computing a canonical key may itself be expensive, and the algorithm is presented as a mathematically justified design, not as a benchmarked improvement. The public repair benchmark currently evaluates ordinary uniform-cost search, not orbit-quotient search.

Table 8: Fresh-name equivalence conditions and failure modes.

Condition	Consequence for quotient search
Roles, dependency endpoints, and history renamed consistently	Structural and rule-relevant information is retained
Costs, goals, and constraints invariant under permitted permutations	Equal-cost transitions and goal outcomes are preserved
Name-dependent action cost or identity-specific prohibition	Renaming is not generally sound; the partition must be rejected
Global capacity or negative condition referencing identities	Must be explicitly checked for equivariance; cannot be ignored

A small executable regression accompanies the manuscript. It tests two permutations of interchangeable replacement labels, each with creation cost two and rewiring cost one, and confirms an optimum of three. It also checks a deliberately identity-dependent cost rule that violates equivariance. These are *unit tests of the assumptions*, not a measured state-space reduction or a realistic service benchmark. Table 8 summarizes the boundary; no runtime or memory improvement is inferred from the regression.

10 Discussion and limitations

The formal model and experiments support three restrained conclusions. First, component creation must be followed by the appropriate relationship changes when the recovery predicate depends on those relationships. Second, independent exact procedures can audit small repair spaces and reveal the cost of alternate repair sequences. Third, structural independence can make cost optimization separable, but the proof fails when a shared constraint changes global feasibility.

Several limitations are decisive. The test instances are synthetic, and the scaling family is deliberately independent. The present search comparison uses standard uniform-cost search, not a new natality-aware reduction. The state representation does not yet address concurrent actions, stochastic failures, partial observability, or real repair logs. In particular, the existing GROOVE executable check does not constitute a matched experiment on this repair model. Related work on graph transformation, model checking, abstraction, and rewriting makes this distinction essential [4, 6, 8, 20, 21, 22].

The fresh-name quotient result specifies a sound abstraction under explicit equivariance assumptions, but its relationship to existing symmetry-reduction algorithms is not a novelty claim. The accompanying regression is a small illustrative check rather than a comparison of full and quotient search. The sufficient footprint certificate is a formal criterion rather than a measured algorithmic improvement; its usefulness and its relation to established independence analyses require further study. A more demanding evaluation would encode the same action costs, graph states, forbidden configurations, and recovery goals in an independently executable graph-transformation grammar. Coupled dependency networks, realistic failure traces, and comparisons against reduction methods would then be necessary to assess whether any natality-specific representation offers a benefit beyond established techniques. The present results should not be read as such a comparison. In particular, Propositions 1 and 2 clarify what creation permits in an expanded action space; neither proposition establishes that graph-transformation tools cannot encode the same behavior.

11 Conclusion

Creation-enabled recovery can be described as an operationally directed state space with explicit identity, admissibility, and path cost. Cost-optimal repair in evolving dependency graphs is naturally expressed as a search over structural changes rather than isolated component fixes. By distinguishing restoration from replacement creation and rewiring, the formulation makes explicit why a newly created component need not immediately restore a dependent service. Exact enumeration and uniform-cost search agree on the minimum cost of a small recovery fixture and on 12 synthetic scaling instances. Under stated noninterference assumptions, the minimum cost of independent repair groups decomposes additively; 24 generated cases validate this implementation, while a shared-capacity counterexample shows why local plan choices cannot always be composed. A conservative local-footprint condition gives a sound way to certify the separability assumptions, but its general implementation and comparative evaluation remain open. Fresh-name equivariance additionally gives a sufficient condition for exact identity-quotient search, with explicit failure cases for identity-sensitive costs and constraints. These findings provide an auditable foundation for further investigation, not evidence of a novel reduction or a matched advantage over existing graph-transformation systems. Establishing those claims will require coupled workloads, independent tool-level comparison, and stronger formal results.

Data and code availability

The original implementation, synthetic fixtures, independent verification programs, and workflow-produced evidence are publicly available at `https://github.com/Arithmetic-Power-Geometry/The-Geometry-of-Natality`. The reproducibility record includes the successful GitHub Actions run `https://github.com/Arithmetic-Power-Geometry/The-Geometry-of-Natality/actions/runs/39151100642`. The additional finite symmetry-regression script is included with the present manuscript package and is not claimed to have been run by the cited repository workflow. The results reported here were generated from synthetic data; no external service logs or real-world datasets were used.

Declarations

Funding: No external funding is reported for this study.
Competing interests: No competing interests are declared.
Data provenance: All evaluated instances are synthetic.

References

[1] Hartmut Ehrig et al. *Fundamentals of Algebraic Graph Transformation*. Monographs in Theoretical Computer Science. An EATCS Series. Springer, 2006. DOI: 10.1007/3-540-31188-2.

[2] Barbara König et al. "A Tutorial on Graph Transformation". In: *Graph Transformation, Specifications, and Nets: In Memory of Hartmut Ehrig*. Ed. by Reiko Heckel and Gabriele Taentzer. Vol. 10800. Lecture Notes in Computer Science. Springer, 2018, pp. 83–104. DOI: 10.1007/978-3-319-75396-6_5.

[3] Arend Rensink. "The GROOVE Simulator: A Tool for State Space Generation". In: *Applications of Graph Transformations with Industrial Relevance*. Springer, 2004, pp. 479–485. DOI: 10.1007/978-3-540-25959-6_40.

[4] Amir Hossein Ghamarian et al. "Modelling and Analysis Using GROOVE". In: *International Journal on Software Tools for Technology Transfer* 14.1 (2012), pp. 15–40. DOI: 10.1007/s10009-011-0186-x.

[5] J. R. Burch et al. "Symbolic Model Checking: 10^{20} States and Beyond". In: *Information and Computation* 98.2 (1992), pp. 142–170. DOI: 10.1016/0890-5401(92)90017-A.

[6] E. Allen Emerson and Thomas Wahl. "Efficient Reduction Techniques for Systems with Many Components". In: *Electronic Notes in Theoretical Computer Science* 130 (2005), pp. 379–399. DOI: 10.1016/j.entcs.2005.03.019.

[7] Arend Rensink, Akos Schmidt, and Dániel Varró. "Model Checking Graph Transformations: A Comparison of Two Approaches". In: *Graph Transformations*. Vol. 3256. Lecture Notes in Computer Science. Springer, 2004, pp. 226–241. DOI: 10.1007/978-3-540-30203-2_17.

[8] Barbara König et al. "Counterexample-Guided Abstraction Refinement for Generalized Graph Transformation Systems". In: *Graph Transformation*. Vol. 15720. Lecture Notes in Computer Science. Springer, 2025, pp. 157–176. DOI: 10.1007/978-3-031-94706-3_8.

[9] M. E. J. Newman. "The Structure and Function of Complex Networks". In: *SIAM Review* 45.2 (2003), pp. 167–256. DOI: 10.1137/S003614450342480.

[10] Detlef Plump. "Term Graph Rewriting". In: *Handbook of Graph Grammars and Computing by Graph Transformation*. Ed. by Hartmut Ehrig et al. Vol. 2. World Scientific Publishing, 1999, pp. 3–61. DOI: 10.1142/9789812815149_0001.

[11] Robert Tarjan. "Depth-First Search and Linear Graph Algorithms". In: *SIAM Journal on Computing* 1.2 (1972), pp. 146–160. DOI: 10.1137/0201010.

[12] David Kempe, Jon Kleinberg, and Éva Tardos. "Maximizing the Spread of Influence through a Social Network". In: *Proceedings of the Ninth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 2003, pp. 137–146. DOI: 10.1145/956750.956769.

[13] George L. Nemhauser, Laurence A. Wolsey, and Marshall L. Fisher. "An Analysis of Approximations for Maximizing Submodular Set Functions—I". In: *Mathematical Programming* 14 (1978), pp. 265–294. DOI: 10.1007/BF01588971.

[14] Kapali P. Eswaran and R. Endre Tarjan. "Augmentation Problems". In: *SIAM Journal on Computing* 5.4 (1976), pp. 653–665. DOI: 10.1137/0205044.

[15] E. Andrew Boyd. “Polyhedral results for the precedence-constrained knapsack problem”. In: *Discrete Applied Mathematics* 41.3 (1993), pp. 185–201. DOI: 10.1016/0166-218X(90)90053-F.

[16] Byungjun You and Takeo Yamada. “A pegging approach to the precedence-constrained knapsack problem”. In: *European Journal of Operational Research* 183.2 (2007), pp. 618–632. DOI: 10.1016/j.ejor.2006.10.031.

[17] Daniel Espinoza, Marcos Goycoolea, and Eduardo Moreno. “The precedence constrained knapsack problem: Separating maximally violated inequalities”. In: *Discrete Applied Mathematics* 194 (2015), pp. 65–80. DOI: 10.1016/j.dam.2015.05.020.

[18] Maribel Fernández and Murdoch J. Gabbay. “Nominal Rewriting with Name Generation: Abstraction vs. Locality”. In: *Proceedings of the 7th ACM SIGPLAN International Symposium on Principles and Practice of Declarative Programming*. ACM, 2005, pp. 47–58. DOI: 10.1145/1069774.1069779.

[19] Amir Hossein Ghamarian and Arend Rensink. “Graph Passing in Graph Transformation”. In: *Electronic Communications of the EASST* 47 (2012). DOI: 10.14279/tuj.eceasst.47.723.

[20] Graham Campbell and Detlef Plump. “Confluence up to Garbage in Graph Transformation”. In: *Theoretical Computer Science* 884 (2021), pp. 1–22. DOI: 10.1016/j.tcs.2021.06.010.

[21] Zsófia Ádám and Zoltán Micskei. “State Space-Based Methods for Validating Model Transformations in Model Checkers”. In: *Software and Systems Modeling* (2026). DOI: 10.1007/s10270-026-01421-6.

[22] Simin Ghasemi et al. “MoTDeReL: Model-Based Testing Through Deep Reinforcement Learning for Software Systems Specified Through Graph Transformation”. In: *Automated Software Engineering* 33 (2026), p. 68. DOI: 10.1007/s10515-026-00610-3.

[23] Roberto Bruni and Arend Rensink. “Experimenting with Reaction Systems Using Graph Transformation and GROOVE”. In: *Natural Computing* 24 (2025), pp. 1075–1099. DOI: 10.1007/s11047-025-10045-6.